

I/O Performance and Scalability Analysis

GSoC Warm-up Task

1. Introduction

The objective of this task is to evaluate the I/O performance of a PyTorch data loading pipeline. The analysis consists of two parts: a local benchmark conducted on GitHub Codespace to measure worker efficiency, and a detailed I/O characterization using Darshan logs from the LCRC cluster.

2. Local Benchmarking (GitHub Codespace)

In this experiment, I generated a 5GB synthetic dataset and tested the training throughput with varying numbers of DataLoader workers.

- 1 Worker: 188.45 samples per second (Baseline)

- 2 Workers: 191.82 samples per second

- 4 Workers: 228.50 samples per second

- 8 Workers: 221.51 samples per second

Observation: While the throughput increased slightly from 1 to 4 workers, the parallel efficiency dropped significantly. At 8 workers, the efficiency fell to 14.69 percent. This indicates that the local storage bandwidth was saturated quickly.

3. HPC Characterization (LCRC Darshan Logs)

Using the Darshan summary reports provided for the LCRC cluster environment, I analyzed the impact of thread counts (1 to 32) on runtime and POSIX I/O performance.

- Thread 1: 244.40 seconds (Baseline)

- Thread 2: 298.64 seconds (Performance regression)

- Thread 4: 209.87 seconds (Peak performance / Sweet spot)

- Thread 8: 215.18 seconds

- Thread 32: 228.32 seconds

Analysis: The data shows a non-linear scaling pattern. The performance regression at 2 threads suggests high overhead in thread management or lock contention. The peak performance at 4 threads corresponds to a POSIX I/O bandwidth of approximately 474 MiB/s, representing the system throughput limit.

4. Cross-Environment Comparison and Conclusion

Comparing the local Codespace results with the LCRC Darshan logs reveals a consistent trend:

Consistency: Both environments demonstrate that increasing concurrency beyond a certain threshold (4 units) leads to diminishing returns or performance degradation.

Bottleneck Identification: The application is clearly I/O-bound. In both cases, the backend storage system reaches its physical limit, and additional workers or threads only increase metadata contention and system overhead.

Engineering Insights: During the local test, I also encountered a disk quota limit (less than 1 percent space remaining), which required manual data management. This real-world constraint, combined with the Darshan log analysis, emphasizes the critical need for efficient I/O orchestration in large-scale machine learning workflows.

5. Future Recommendations

To further optimize performance, future work should explore:

Using advanced data formats like HDF5 or WebDataset to reduce small-file metadata overhead.

Implementing data pre-fetching and larger I/O buffers to mitigate the effects of thread contention.

Attached the code execution results as your reference